CODEFLOW / PSUT

# HTML and CSS Introduction

In-class guide for beginners.
Type along, refresh often, and break things on purpose.

# Before we start

This is the handout for the in-class introduction to **HTML** and **CSS**. It is designed to be followed live: we read a little, type a little, refresh the browser, and see what happened. You will build your own project from your prepared project files; here we practise the ideas in a small throwaway page so nothing gets in the way.

| Time | Part | What we do |
|---|---|---|
| 10 min | 1. Setup | Make a practice folder, open it in VS Code, open the page in a browser. |
| 25 min | 2. HTML | Tags, headings, paragraphs, links, images, lists, page structure. |
| 25 min | 3. CSS | Selectors, colors, fonts, spacing, the box model, classes. |
| 10 min | 4. Put it together | Style the HTML page, then do the mini challenges. |
| - | 5. Cheat sheet | Reference to keep next to you while you work on your project. |

**Teacher tip (for the live session)**

Type every example live and let students predict what will happen before you refresh. Break something on purpose (delete a closing tag, misspell a color) and fix it together. Mistakes are the best teacher.

# Part 1 - Setup

1. Create a folder on your Desktop called `practice`.
2. Open VS Code, choose **File > Open Folder**, and select it.
3. Create a new file named `index.html` and another named `style.css`. The names matter, and there must be no spaces.
4. To see your page, double-click `index.html` in the file explorer so it opens in your browser. After every change in VS Code: **save (Ctrl+S)** and **refresh the browser (F5)**.

**Key idea**

A website is just text files. The browser reads them and draws the page. **index.html** is the traditional name of the main page of a site.

# Part 2 - HTML: the structure

**Think first**

How does a browser know what is a heading, what is a paragraph and what is a button?

## 2.1 Tags and elements

HTML uses **tags** to label content. Most tags come in pairs: an **opening tag** and a **closing tag** (with a slash). The whole thing is an **element**.

```
<p>This is a paragraph.</p>
 ^        ^               ^
 opening  content         closing
```

Rules to remember: tags are written inside angle brackets; the closing tag has a `/`; tag names are lowercase. Some tags have no content and no closing tag, such as `<img>` and `<br>`.

## 2.2 The skeleton of every HTML page

Every page starts with the same skeleton. Do not worry about memorising it: you will copy it each time, and your editor can type it for you (in VS Code, type `!` and press Tab).

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My First Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <!-- everything people SEE goes here -->

</body>
</html>
```

| Part | Meaning |
|---|---|
| <!DOCTYPE html> | Tells the browser: "this is a modern HTML page". |
| <html lang="en"> | The root. Everything lives inside it. `lang` says the page is in English. |
| <head> | Information *about* the page. Not displayed in the page area. |
| <meta charset> | Lets the page show letters correctly, including Arabic. |
| <meta viewport> | Makes the page behave properly on phones. |
| <title> | The text on the browser tab. |
| <link rel="stylesheet"> | Connects your CSS file to this page. Without it, no styles. |
| <body> | Everything visible: text, images, buttons. |

## 2.3 Headings and paragraphs

```
<h1>My Name</h1>        <!-- the main title, use once -->
<h2>About me</h2>       <!-- a section title -->
<h3>A smaller title</h3>
<p>I am learning to build websites.</p>
<p>This is a second paragraph.</p>
```

Headings go from `h1` (most important) to `h6`. Choose them for *meaning*, not size: you can change sizes later with CSS. Use one `h1` per page.

## 2.4 Links

```
<a href="https://github.com">Visit my GitHub</a>
```

The a (anchor) tag makes a link. `href` is an **attribute**: extra information inside the opening tag, written as `name="value"`. The text between the tags is what people click.

## 2.5 Images

```
<img src="profile.jpg" alt="A photo of me smiling">
```

- `src` is the image file's location. If the image is in the same folder as the page, its name is enough.
- `alt` describes the image for people who cannot see it (screen readers) and shows if the image fails to load. Always write it.
- `img` has no closing tag.

> **If the image does not show**
>
> Check three things: the file is in the right folder; the name matches *exactly* (including capitals and the extension, such as .jpg vs .jpeg); no spaces in the name.

## 2.6 Lists

```
<ul>                        <ol>
  <li>HTML</li>               <li>Plan</li>
  <li>CSS</li>                <li>Build</li>
  <li>Teamwork</li>           <li>Test</li>
</ul>                       </ol>
```

`ul` is an unordered (bulleted) list, `ol` is an ordered (numbered) list, and each item is an `li`.

## 2.7 Grouping and buttons

```
<main class="profile">      <!-- the main part of the page -->
  <h1>Hello!</h1>
  <button>Click Me</button>
</main>
```

Some tags exist to **group** things so CSS can style them together: `div` (a generic box), `main`, `header`, `footer`, `section`. Tags with a clear meaning like these are called **semantic**: they tell browsers and search engines what the content is.

## 2.8 Nesting and indentation

Elements can sit inside other elements. This is called **nesting**. The rule: **close the one you opened last, first**.

```
RIGHT:  <ul><li>Item</li></ul>
WRONG:  <ul><li>Item</ul></li>
```

Indent (push right) the children of every element by two spaces. It does not change how the page looks, but it lets you see the structure at a glance and spot a missing closing tag.

### Practice A (5 minutes)

In `index.html`, create a page with: a title, a paragraph about you, a list of three hobbies, a link to a website you like, and an image. Refresh after each addition.

# Part 3 - CSS: the style

> **Think first**
>
> Right now our page looks plain. What decides colors, fonts and spacing?

**CSS** stands for Cascading Style Sheets. It does not add content; it describes how existing HTML elements should look. HTML = what is there. CSS = how it looks.

## 3.1 The shape of a CSS rule

```
h1 {
  color: rebeccapurple;
  font-size: 32px;
}
^      ^              ^
selector  property    value
```

- The **selector** says *which* elements to style (here: every `h1`).
- Inside curly braces `{ }` you write **declarations**, each a **property** and a **value**.
- A colon `:` goes between them and a semicolon `;` ends each one. Forgetting the semicolon is the number one beginner mistake.

## 3.2 Connecting CSS to the page

Because of the `<link rel="stylesheet" href="style.css">` line in the head, everything in `style.css` applies to `index.html`. If your styles do nothing, check this line and the file name first.

## 3.3 Colors

| Way to write it | Example | Notes |
|---|---|---|
| Name | red, white, rebeccapurple | About 140 names exist. |
| Hex code | #6941c6 | A # then six characters. Copy them from a color picker. |
| RGB | rgb(105, 65, 198) | Amounts of red, green, blue from 0 to 255. |

```
body {
  background: #f3f0ff;   /* background color */
  color: #1f1637;        /* text color */
}
```

Tip: in VS Code, writing a hex code shows a small color swatch you can click to open a picker.

## 3.4 Text and fonts

```
p {
  font-family: Arial, sans-serif;   /* the font; list a backup */
  font-size: 18px;                  /* the size */
  font-weight: bold;                /* thickness */
  text-align: center;               /* left, center or right */
  line-height: 1.6;                 /* space between lines */
}
```

## 3.5 The box model: how spacing works

**Every element is a rectangular box.** This is the most important idea in CSS layout. A box has four layers, from inside out:

| Layer | What it is | Property |
|---|---|---|
| Content | The text or image itself. | width, height |
| Padding | Space between the content and the border, *inside* the box. | padding |
| Border | A line around the padding. | border |
| Margin | Space *outside* the box, pushing other things away. | margin |

Think of a framed picture on a wall: the picture is the content, the white mat inside the frame is the padding, the frame is the border, and the gap to the next picture on the wall is the margin.

```
div {
  width: 300px;
  padding: 20px;                  /* space inside */
  border: 2px solid #6941c6;      /* thickness, style, color */
  margin: 16px;                   /* space outside */
  border-radius: 12px;            /* rounded corners */
}
```

Two handy tricks: `margin: auto;` together with a `max-width` centers a box horizontally, and `border-radius: 50%` on a square image makes it a circle.

## 3.6 Classes: styling just some elements

Styling by tag name (p) changes *all* paragraphs. To style only some, give them a **class** in HTML and select it in CSS with a dot.

*HTML (top) and CSS (bottom)*

```
<p class="highlight">Special text</p>
<p>Normal text</p>

/* in style.css */
.highlight {
  background: yellow;
  font-weight: bold;
}
```

**Remember the dot**

p selects tags. `.highlight` selects the class named highlight. In HTML you write `class="highlight"` **without** the dot; in CSS you write it **with** the dot.

### 3.7 Hover effects

```
button {
  background: #6941c6;
  color: white;
  border: none;
  padding: 10px 20px;
  border-radius: 8px;
}

button:hover {            /* when the mouse is over it */
  background: #4c2a99;
}
```

`:hover` applies the style only while the mouse is over the element. It is a small touch that makes a page feel alive.

### 3.8 Why "cascading"?

When two rules style the same thing, the browser must choose. Roughly: a more **specific** selector wins (a class beats a tag), and if they are equally specific, the rule written **later** wins. You do not need more than that for now. If a style is not working, another rule may be overriding it: use your browser's DevTools (press F12 and click the element) to see which rules apply.

### Practice B (10 minutes)

1. Give the body a soft background color and a different text color.
2. Make the h1 a new color and center it.
3. Give the image a circular shape, using width, height and border-radius: 50%.
4. Create a class called `card` with padding, a border, rounded corners and a max-width, and add it to a div around your content.

## Part 4 - Put it together

Here is a complete small example. Type it live, refresh, then change things. It is deliberately different from your project so you build the idea, not just copy.

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My Favorite Place</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="card">
    <h1>Petra</h1>
    <p class="tagline">The rose-red city</p>
    <p>One of the most famous places in Jordan.</p>
    <ul>
      <li>Walk through the Siq</li>
      <li>See the Treasury</li>
    </ul>
    <a href="https://www.visitjordan.com">Plan a visit</a>
  </div>
</body>
</html>
```

```
/* style.css */
body {
  font-family: Arial, sans-serif;
  background: #f3f0ff;
  padding: 40px 16px;
}

.card {
  background: white;
  max-width: 420px;
  margin: auto;
  padding: 28px;
  border-radius: 16px;
}

h1 {
  color: #6941c6;
}

.tagline {
  color: #777;
  font-style: italic;
}

a {
  color: #6941c6;
  font-weight: bold;
}
```

## Mini challenges

| Level | Challenge | Hint |
|---|---|---|
| Easy | Change the card color to light pink and the h1 to dark red. | background and color properties. |
| Easy | Add a third list item. | Copy an li line. |
| Medium | Add a photo of the place above the title and make it fit the card width. | img { width: 100%; } |
| Medium | Make the link look like a button. | padding, background, color, border-radius, and text-decoration: none. |
| Stretch | Make the link darker when the mouse is over it. | a:hover { ... } |

### When something does not work

1. Did you **save** both files and **refresh** the browser?
2. Look for a missing closing tag, a missing >, or a missing quote.
3. In CSS: is there a `:` and a `;` on every line, and are the curly braces matched?
4. Is the file name and `href` spelled exactly the same (capitals matter)?
5. Open DevTools with F12 to inspect the element and see which styles apply.
6. Undo the last change you made and try again. Change one thing at a time.

# Part 5 - Cheat sheet

### Common HTML tags

| Tag | Use | Example |
|---|---|---|
| <h1> ... <h6> | Headings | <h1>Title</h1> |
|  | Paragraph | Text |
| <a> | Link | <a href="url">Click</a> |
| <img> | Image | <img src="a.jpg" alt="..."> |
| <ul> <ol> <li> | Lists | <ul><li>One</li></ul> |
|  | Generic grouping box | ... |
| <button> | Button | <button>Go</button> |
| <br> | Line break | Line one<br>Line two |

### Common CSS properties

| Property | Does | Example value |
|---|---|---|
| color | Text color | #6941c6 |

| Property | Does | Example value |
|---|---|---|
| background | Background color | #f3f0ff |
| font-family | Font | Arial, sans-serif |
| font-size | Text size | 18px |
| font-weight | Boldness | bold |
| text-align | Alignment | center |
| width / max-width | Size | 420px |
| padding | Space inside | 20px |
| margin | Space outside | 16px or auto |
| border | Line around | 2px solid #6941c6 |
| border-radius | Rounded corners | 12px or 50% |

## Where to keep learning

- **MDN Web Docs** (developer.mozilla.org) - the most reliable reference for HTML and CSS.
- **Your browser's DevTools** (F12) - change styles live and see the result instantly.
- **Your own project** - the best teacher. Change a value, refresh, observe.